

Single-layer NN/Shallow NN

Regression and Classification

Fundamentals of Deep Learning · Class 6 · Classification

4 August

Outline

From Regression to Classification

Linear Classifiers and Decision Boundaries

The Maximum-Likelihood Recipe

Binary Classification

Multi-class Classification

The Bigger Picture

From Regression to Classification

What changes

	Regression (Class 5)	Classification (today)
Target	$t \in \mathbb{R}$	$t \in \{0, 1\}$ or one of K classes
Output act.	identity	sigmoid / softmax
Loss	sum of squares	cross-entropy
Prediction	a value	a probability

The hidden layer is **unchanged**: same architecture, same learned features $z_j = h(a_j)$.

ML linear regression $\rightarrow$ logistic regression

NN NN regression $\rightarrow$ NN classification —
same translation

Recap: the shallow network

From Class 5 (Bishop & Bishop, 2024, Eqs. 6.1–6.3):

$$a_j = \sum_i w_{ji}^{(1)} x_i + w_{j0}^{(1)}, \quad z_j = h(a_j), \quad a_k = \sum_j w_{kj}^{(2)} z_j + w_{k0}^{(2)}$$

- Regression: $y_k = a_k$ — the output *is* the pre-activation
- Classification: pass a_k through an **output activation** that produces a probability
- Everything before the output activation is identical

ML $\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})$, then squash

NN $\sum_j w_{kj}^{(2)} z_j + w_{k0}^{(2)}$, then squash — features learned

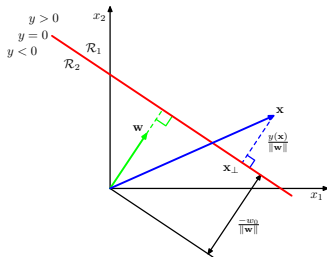
Linear Classifiers and Decision Boundaries

Discriminant functions

The simplest classifier (Bishop & Bishop, 2024, §5.1.1):

$$y(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0$$

- Assign $\mathcal{C}_1$ if $y(\mathbf{x}) \geq 0$, else $\mathcal{C}_2$
- Boundary $y(\mathbf{x}) = 0$: a hyperplane with normal $\mathbf{w}$, offset set by w_0

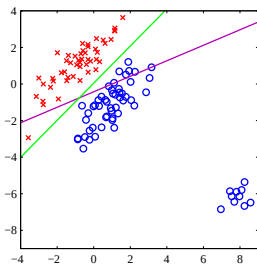
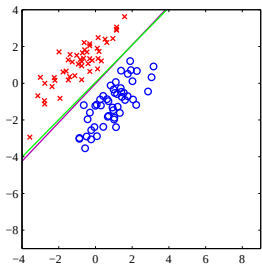


Bishop & Bishop (2024), Fig. 5.1.

ML decision boundary: a hyperplane in input space

NN a hyperplane in *learned feature space* — nonlinear in $\mathbf{x}$

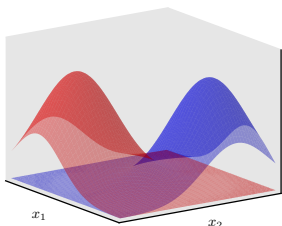
Why not just do least squares?



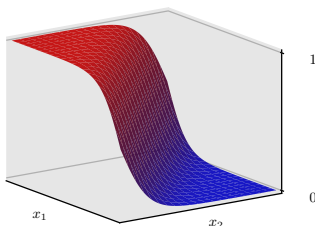
Bishop & Bishop (2024), Fig. 5.4: least squares (magenta) vs. logistic regression (green).

- Adding distant, *correctly classified* points drags the least-squares boundary — they are penalised for being “too right”
- Squared error is the wrong loss for targets in $\{0, 1\}$: we need a loss built for probabilities

Where the sigmoid comes from



class-conditionals $p(\mathbf{x} | \mathcal{C}_k)$ — B&B (2024), Fig. 5.13



posterior $p(\mathcal{C}_1 | \mathbf{x})$ — a sigmoid

- Gaussian classes, shared covariance $\Rightarrow$ posterior is **exactly** $\sigma(\mathbf{w}^T \mathbf{x} + w_0)$ Bishop & Bishop, §5.3
- The sigmoid is the **natural form of a posterior probability**, not an arbitrary squashing choice

The Maximum-Likelihood Recipe

The recipe for a loss function

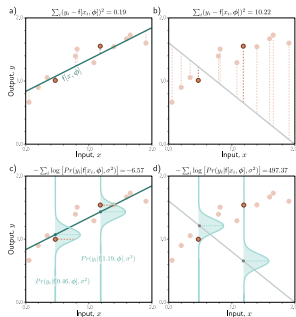
The network predicts, for each input, the parameters of a probability distribution over the output (Prince, 2023, §5.1–5.2):

1. Choose a distribution $Pr(y \mid \theta)$ suited to the domain of y
2. Let the network predict its parameters: $\theta = f[x, \phi]$
3. Train by minimising the negative log-likelihood:

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[- \sum_i \log Pr(y_i \mid f[x_i, \phi]) \right]$$

- Log: products of small numbers $\rightarrow$ sums; same maximum (log is monotonic)
- Inference: report the peak of the predicted distribution

You already know one instance



Prince (2023), Fig. 5.4.

Class 5, rewritten as the recipe:

- Domain $y \in \mathbb{R} \rightarrow$ choose a **Gaussian**
- Network predicts its mean $\mu = f[x, \phi]$
- Negative log-likelihood = **least squares**

ML Gaussian noise $\rightarrow$ squared error (B&B §4.1.1)

NN today: Bernoulli $\rightarrow$ cross-entropy, same recipe

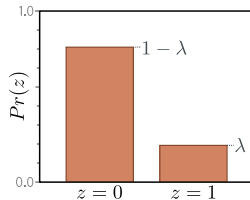
Binary Classification

Step 1: the Bernoulli distribution

Domain: $y \in \{0, 1\}$. One parameter $\lambda \in [0, 1]$:

$$Pr(y | \lambda) = \lambda^y (1 - \lambda)^{1-y}$$

- $Pr(y=1) = \lambda$, $Pr(y=0) = 1 - \lambda$
- The network must predict λ — a number in $[0, 1]$
- But network outputs are **arbitrary real values...**



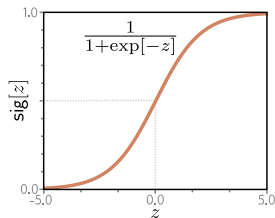
Prince (2023), Fig. 5.6.

Step 2: the logistic sigmoid

Map the real line to $[0, 1]$ (Bishop & Bishop, 2024, Eq. 5.42):

$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$

- $\sigma(0) = \frac{1}{2}$; symmetry $\sigma(-a) = 1 - \sigma(a)$
- Derivative $\sigma'(a) = \sigma(a)(1 - \sigma(a))$ Eq. 5.72
- So: $\lambda = \sigma(f[x, \phi])$ – a valid probability

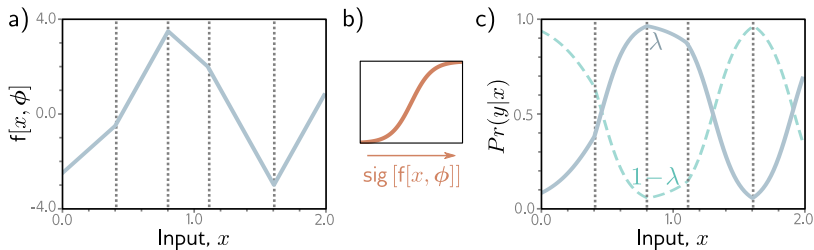


Prince (2023), Fig. 5.7.

ML logistic regression: $\sigma(\mathbf{w}^\top \phi(\mathbf{x}))$

NN same sigmoid on a *learned-feature* read-out

The binary classification model



Prince (2023), Fig. 5.8: (a) network output; (b) after the sigmoid: probability $\lambda(x)$; (c) likelihood per point.

Step 3: binary cross-entropy

Negative log-likelihood of the Bernoulli model, with $y_n = \sigma(a_n)$ read as $p(t_n=1 \mid \mathbf{x}_n)$ (Bishop & Bishop, 2024, Eq. 5.74):

$$E(\mathbf{w}) = - \sum_{n=1}^N \left[t_n \ln y_n + (1 - t_n) \ln(1 - y_n) \right]$$

- $t_n=1$ penalises small y_n ; $t_n=0$ penalises large y_n
- Gradient (Eq. 5.75): $\nabla E = \sum_n (y_n - t_n) \phi_n$ — the **same $y - t$ form** as least squares
- Non-convex with a network inside; convex in the output layer alone

ML logistic regression: cross-entropy, convex

NN same loss; non-convex in all parameters

Why not squared error on probabilities?

With a sigmoid output and squared error:

$$\frac{\partial E_{\text{SSE}}}{\partial a_n} = (y_n - t_n) \sigma'(a_n), \quad \frac{\partial E_{\text{CE}}}{\partial a_n} = y_n - t_n$$

- Saturated output ($y_n \approx 0$ or 1): $\sigma'(a_n) \approx 0 \rightarrow$ the SSE **gradient vanishes** even when the prediction is wrong
- Cross-entropy has no σ' factor — learning does not stall
- This is the recipe at work: **match the loss to the output distribution**

Distribution $\rightarrow$ loss. Gaussian $\rightarrow$ squared error; Bernoulli $\rightarrow$ cross-entropy.

Multi-class Classification

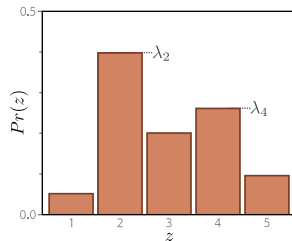
Step 1: the categorical distribution

Domain: $y \in \{1, \dots, K\}$. Parameters

$\lambda_1, \dots, \lambda_K$:

$$\Pr(y=k) = \lambda_k, \quad \lambda_k \geq 0, \quad \sum_k \lambda_k = 1$$

- The network must predict K numbers that are positive and sum to one
- K raw outputs $a_1, \dots, a_K$ are unconstrained...



Prince (2023), Fig. 5.9.

Step 2: softmax

Normalise the K pre-activations (Bishop & Bishop, 2024, Eqs. 5.76–5.77):

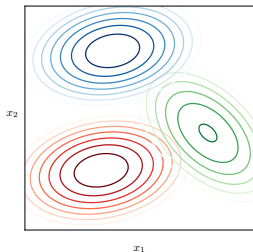
$$y_k = \text{softmax}_k(\mathbf{a}) = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)}$$

- $y_k > 0$ and $\sum_k y_k = 1$ — a valid categorical parameter vector
- $K=2$ recovers the sigmoid: $\text{softmax on } (a_1, a_2) = \sigma(a_1 - a_2)$
- Adding a constant to every a_k changes nothing (used for numerical stability)

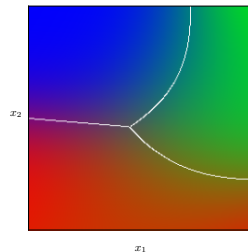
ML multinomial (softmax) logistic regression

NN softmax on K learned-feature read-outs

Softmax posteriors, visualised



three Gaussian classes

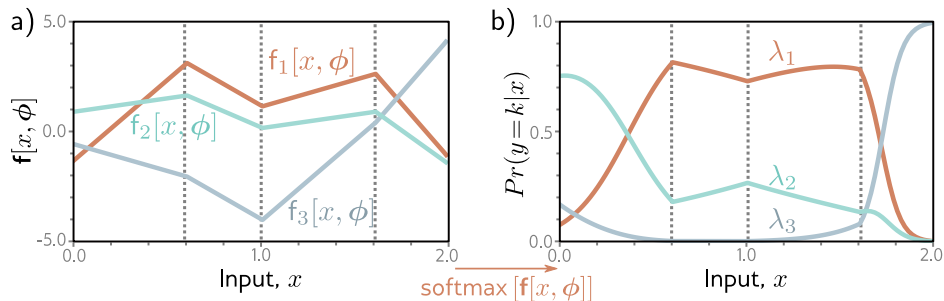


posterior class probabilities (RGB)

Bishop & Bishop (2024), Fig. 5.14.

- Softmax posteriors partition input space; colours blend where classes overlap
- Decision boundaries: where the largest two posteriors are equal

The multi-class model



Prince (2023), Fig. 5.10 ($K=3$): (a) three piecewise-linear outputs; (b) after softmax — non-negative, sum to one; (c) decision regions. Shared hidden units $\Rightarrow$ joints in the same places.

Step 3: multi-class cross-entropy

One-hot targets $\mathbf{t}_n \in \{0, 1\}^K$, $\sum_k t_{nk} = 1$ (Bishop & Bishop, 2024, Eq. 5.80):

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk}$$

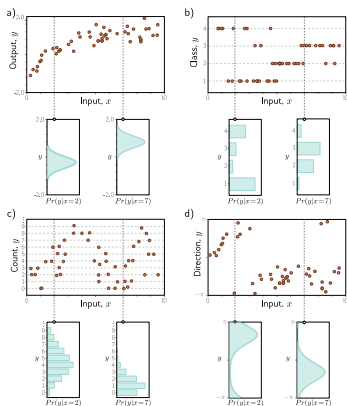
- Only the true class contributes: maximise the log-probability the model assigns to it
- Gradient (Eq. 5.81): $\partial E / \partial a_{nk} = y_{nk} - t_{nk} = \mathbf{y} - \mathbf{t}$ again
- Reduces to binary cross-entropy for $K=2$

ML softmax regression: convex in $\mathbf{w}$

NN network inside: non-convex, trained by gradient descent

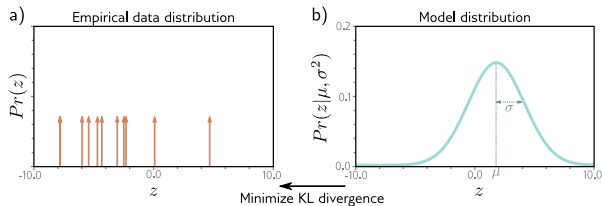
The Bigger Picture

One recipe, any output type



Prince (2023), Fig. 5.11: choose the distribution to match the prediction domain — univariate/multivariate, continuous/discrete, circular, bounded, count data.

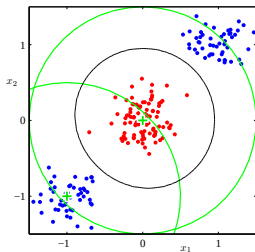
Cross-entropy: the second view



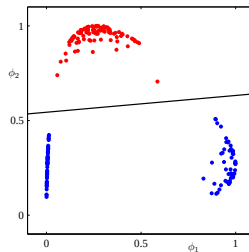
Prince (2023), Fig. 5.12.

- Minimising negative log-likelihood $\equiv$ minimising the **KL divergence** between the empirical data distribution and the model distribution
- The remaining term is the **cross-entropy** — hence the loss's name Prince, §5.7

Fixed features vs. learned features, again



input space: nonlinear boundary



feature space (ϕ_1, ϕ_2) : linear boundary

Bishop & Bishop (2024), Fig. 5.15; cf. Fig. 5.16 for the network view.

ML fixed Gaussian ϕ linearise the problem

NN a shallow network *learns* the features that linearise it

Output activations: the complete table

Task	Distribution	Output activation	Loss
Regression	Gaussian	identity	sum of squares
Binary cl.	Bernoulli	sigmoid $\sigma(a)$	binary cross-entropy
Multi-class	categorical	softmax	cross-entropy

In all three cases the output-layer gradient is $y_n - t_n$: prediction minus target.

Bishop & Bishop (2024), §5.4, §6.1; Prince (2023), §5.3–5.5.

The dictionary (Classes 5–6)

	Linear/Basis-Function Model	Shallow Neural Network
Features	$\phi_j(\mathbf{x})$, fixed	$z_j = h(a_j)$, learned
Regression	$\mathbf{w}^\top \phi$; SSE; normal equations	linear read-out; SSE; gradient descent
Binary cl.	$\sigma(\mathbf{w}^\top \phi)$; CE; convex	$\sigma(\text{read-out})$; CE; non-convex
Multi-class	softmax; CE; convex	softmax; CE; non-convex
Boundary	hyperplane in feature space	hyperplane in <i>learned</i> feature space

Classification = the Class-5 network with a new output activation and the matching loss.

Takeaways

1. The loss follows from a **distribution over outputs**: pick it to match the domain, predict its parameters, minimise negative log-likelihood
2. Binary: Bernoulli $\rightarrow$ sigmoid $\rightarrow$ binary cross-entropy; the sigmoid is the natural form of a **posterior probability**
3. Multi-class: categorical $\rightarrow$ softmax $\rightarrow$ cross-entropy
4. Every case has output gradient $y_n - t_n$; matching loss to distribution avoids saturation

Class 7: stacking hidden layers — **deep networks** and why depth helps.

References

- C. M. Bishop & H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. §5.1, §5.3–5.4, §6.1. Figures 5.1, 5.4, 5.13, 5.14, 5.15 from the authors' published figure set, with attribution.
- S. J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. Ch. 5. Figures 5.4, 5.6–5.12 and teaching sequence from the author's repository (<https://udlbook.github.io/udlbook/>), CC BY-NC-ND, with attribution.

Questions?